

AI 구매 Agent 상품 데이터 수집 시스템 개발 계획서

1. 프로젝트 개요

1.1 프로젝트명

AgentCart 상품 데이터 수집 시스템

1.2 개발 목적

AI 구매 Agent가 사용자의 구매 요청에 적합한 상품을 탐색하고 비교·검증할 수 있도록, 여러 온라인 쇼핑몰에서 상품 정보를 안정적으로 수집하는 시스템을 개발한다.

수집 대상 정보는 상품명, 가격, 판매처, 상세 설명, 이미지, 옵션, 재고 상태, 배송 정보, 리뷰 및 평점 등으로 구성한다.

수집된 데이터는 공통 상품 데이터 구조로 정규화한 후 AI Agent의 상품 검색, 비교, 추천 및 구매 지원 기능에 활용한다.

2. 개발 배경

온라인 쇼핑몰은 사이트마다 상품 정보를 제공하는 방식이 다르다.

일부 사이트는 서버에서 완성된 HTML을 반환하지만, 일부 사이트는 JavaScript 실행 이후 상품 정보를 화면에 표시한다. 또한 상품 데이터를 화면 HTML에 직접 포함하지 않고 내부 API를 통해 JSON 형태로 전달하는 사이트도 존재한다.

따라서 하나의 수집 방식만 사용하는 것이 아니라, 대상 사이트의 구조에 따라 다음 방식을 선택적으로 적용한다.

1. HTML 정적 파싱
2. 네트워크 API의 JSON 데이터 수집
3. Headless Browser 기반 동적 페이지 수집
4. 공식 API 또는 제휴 API 연동

3. 수집 대상

3.1 수집 사이트

초기 개발 단계에서는 수집 난이도와 상품 데이터 구조를 확인하기 위해 제한된 쇼핑몰을 대상으로 PoC를 진행한다.

예시 대상:

- 종합 온라인 쇼핑몰
- 브랜드 공식 쇼핑몰
- 가격 비교 사이트
- 전문 상품 판매 사이트
- 해외 전자상거래 사이트

각 사이트는 서비스 정책, robots.txt, 접근 제한, API 제공 여부를 먼저 확인한 후 수집 대상으로 선정한다.

3.2 수집 데이터

구분	수집 항목
상품 기본 정보	상품명, 상품 ID, 브랜드, 카테고리
가격 정보	정상가, 판매가, 할인율, 쿠폰 적용 여부
판매 정보	판매처, 판매자, 상품 URL
옵션 정보	색상, 크기, 모델, 옵션별 가격
재고 정보	판매 가능 여부, 품절 여부
배송 정보	배송비, 배송 예정일, 배송 방식
평가 정보	평점, 리뷰 수, 리뷰 내용
이미지 정보	대표 이미지, 상세 이미지
상세 정보	상품 설명, 스펙, 주의사항
수집 메타데이터	수집 시간, 수집 방식, 원본 URL

4. 크롤링 방식

4.1 HTML 파싱 방식

서버가 상품 정보가 포함된 HTML을 직접 반환하는 경우 사용하는 방식이다.

처리 과정

1. 대상 URL에 HTTP 요청을 전송한다.
2. HTML 응답을 수신한다.
3. CSS Selector 또는 XPath로 필요한 요소를 찾는다.
4. 상품명, 가격, 이미지 등의 데이터를 추출한다.

5. 추출 데이터를 공통 데이터 구조로 변환한다.

적용 기술 예시

- Python Requests
- BeautifulSoup
- lxml
- Scrapy

장점

- 구현이 단순하다.
- 처리 속도가 빠르다.
- 브라우저를 실행하지 않아 서버 자원 사용량이 적다.
- 대량 수집에 상대적으로 유리하다.

단점

- JavaScript 실행 후 표시되는 데이터는 수집하기 어렵다.
- HTML 구조가 변경되면 Selector가 동작하지 않을 수 있다.
- 사이트별로 별도의 파싱 규칙이 필요하다.

4.2 JSON API 수집 방식

브라우저 개발자 도구의 Network 탭에서 상품 데이터를 반환하는 API 요청을 확인하고, 해당 API를 직접 호출하는 방식이다.

처리 과정

1. 브라우저 개발자 도구를 실행한다.
2. Network 탭에서 Fetch/XHR 요청을 확인한다.
3. 상품 목록 또는 상세 정보를 반환하는 요청을 찾는다.
4. 요청 URL, HTTP Method, Query Parameter, Header 및 Body를 분석한다.
5. 동일한 요청을 크롤러에서 재현한다.
6. JSON 응답에서 필요한 필드를 추출한다.

적용 기술 예시

- Python Requests
- HTTPX
- aiohttp
- Scrapy

장점

- HTML 파싱보다 데이터 구조가 명확한 경우가 많다.
- 상품 목록, 페이지네이션, 옵션 데이터를 효율적으로 수집할 수 있다.
- 불필요한 화면 요소를 제외하고 필요한 데이터만 가져올 수 있다.
- 브라우저 자동화보다 일반적으로 빠르고 자원 사용량이 적다.

단점

- 인증 토큰, 쿠키, 서명 값 등이 필요할 수 있다.
- 내부 API 구조가 예고 없이 변경될 수 있다.
- API 호출이 세션이나 브라우저 환경에 종속될 수 있다.
- 공개 API가 아닌 내부 엔드포인트는 이용 정책과 접근 조건을 별도로 검토해야 한다.

4.3 Headless Browser 방식

JavaScript 실행 이후에 상품 정보가 생성되거나, 사용자 동작이 필요한 경우 브라우저를 자동화하여 수집하는 방식이다.

처리 과정

1. 자동화 브라우저로 대상 페이지에 접속한다.
2. JavaScript와 화면 렌더링이 완료될 때까지 대기한다.
3. 스크롤, 클릭, 페이지 이동 등의 동작을 수행한다.
4. 렌더링된 DOM 또는 네트워크 응답에서 데이터를 추출한다.
5. 추출한 데이터를 공통 데이터 구조로 변환한다.

적용 기술 예시

- Playwright
- Selenium
- Puppeteer

장점

- 실제 사용자가 보는 화면과 유사한 환경에서 데이터를 수집할 수 있다.
- 무한 스크롤, 팝업, 옵션 선택 등 사용자 상호작용을 처리할 수 있다.
- 동적 페이지에 대응할 수 있다.
- 브라우저에서 발생하는 네트워크 응답을 직접 확인할 수 있다.

단점

- CPU와 메모리 사용량이 크다.

- 처리 속도가 상대적으로 느리다.
- 타임아웃, 렌더링 지연 및 브라우저 오류가 발생할 수 있다.
- 대규모 병렬 처리 시 운영 비용이 증가한다.

4.4 공식 API 및 제휴 API 방식

대상 플랫폼에서 상품 검색 API, 판매자 API 또는 제휴 API를 제공하는 경우 우선적으로 검토한다.

장점

- 데이터 구조가 비교적 안정적이다.
- 이용 범위와 호출 정책이 명확하다.
- HTML 변경의 영향을 적게 받는다.
- 상품 ID, 가격, 재고 등 정형 데이터를 확보하기 쉽다.

단점

- API 신청 및 승인이 필요할 수 있다.
- 호출량 제한이 존재할 수 있다.
- 일부 상품 정보만 제공할 수 있다.
- 사용 비용이나 제휴 조건이 존재할 수 있다.

5. 수집 방식 선택 기준

각 대상 사이트에 대해 다음 우선순위로 수집 방식을 결정한다.

1순위: 공식 API

공식 API 또는 제휴 API가 제공되며 프로젝트 목적에 맞게 사용할 수 있는 경우 이를 우선 적용한다.

2순위: JSON API

상품 데이터가 Network 요청을 통해 JSON으로 전달되고, 해당 요청을 안정적으로 재현할 수 있는 경우 적용한다.

3순위: HTML 파싱

초기 HTML에 필요한 상품 정보가 포함되어 있는 경우 적용한다.

4순위: Headless Browser

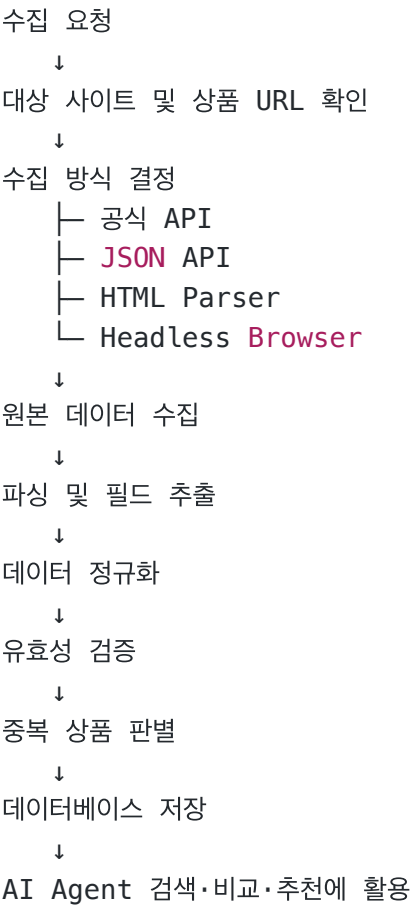
JavaScript 렌더링이나 사용자 상호작용 없이는 데이터에 접근하기 어려운 경우에만 적용한다.

판단표

판단 항목	공식 API	JSON API	HTML 파싱	브라우저 자동화
처리 속도	높음	높음	높음	낮음
구현 난이도	중간	중간	낮음	높음
자원 사용량	낮음	낮음	낮음	높음
구조 안정성	높음	중간	낮음	중간
동적 페이지 대응	높음	높음	낮음	높음
운영 비용	낮음	낮음	낮음	높음

6. 시스템 구성

6.1 전체 처리 흐름



6.2 주요 컴포넌트

Crawl Scheduler

사이트별 수집 주기와 실행 시간을 관리한다.

URL Frontier

수집 예정 URL과 수집 완료 URL을 구분해 관리한다.

Site Adapter

사이트별 수집 로직을 분리한다.

```
adapters/  
├─ site_a/  
│   ├── collector.py  
│   ├── parser.py  
│   └── mapper.py  
├─ site_b/  
│   ├── collector.py  
│   ├── parser.py  
│   └── mapper.py  
└─ common/
```

Downloader

HTTP 요청 또는 브라우저 실행을 통해 원본 데이터를 가져온다.

Parser

HTML 또는 JSON 응답에서 필요한 상품 필드를 추출한다.

Normalizer

사이트마다 다른 상품 데이터 구조를 공통 형식으로 변환한다.

Validator

필수 필드 누락, 가격 형식 오류, 잘못된 URL 및 비정상 데이터를 검사한다.

Deduplicator

동일 상품이 여러 판매처 또는 여러 URL에서 수집되는 경우 중복 여부를 판별한다.

Storage

원본 데이터, 정규화 데이터 및 수집 이력을 저장한다.

7. 데이터 구조

7.1 공통 상품 데이터 예시

```
{
  "source": "shop_a",
  "source_product_id": "A123456",
  "product_name": "무선 노이즈 캔슬링 헤드폰",
  "brand": "Example",
  "category": "electronics/audio/headphone",
  "product_url": "https://example.com/products/A123456",
  "image_url": "https://example.com/images/A123456.jpg",
  "original_price": 329000,
  "sale_price": 279000,
  "currency": "KRW",
  "rating": 4.7,
  "review_count": 3251,
  "stock_status": "IN_STOCK",
  "shipping_fee": 0,
  "options": [
    {
      "name": "color",
      "value": "black",
      "additional_price": 0
    }
  ],
  "collected_at": "YYYY-MM-DDTHH:mm:ss+09:00"
}
```

7.2 원본 데이터 보관

사이트 구조 변경이나 파싱 오류에 대응하기 위해 일정 기간 원본 응답을 보관한다.

원본 데이터 유형은 다음과 같다.

- HTML 원문
- JSON 응답
- HTTP 상태 코드
- 응답 헤더
- 요청 URL
- 수집 시각
- 파서 버전

8. 상품 중복 판별

동일 상품이 다른 URL이나 판매처에서 반복 수집될 수 있으므로 다음 정보를 조합해 중복 여부를 판단한다.

- 사이트 상품 ID
- 모델명
- 브랜드
- 표준 상품 코드
- 상품명 정규화 결과
- 주요 스펙
- 대표 이미지 해시

중복 판별 수준은 다음과 같이 구분한다.

유형	설명
URL 중복	동일 URL이 반복 수집된 경우
콘텐츠 중복	URL은 다르지만 응답 내용이 같은 경우
상품 중복	판매처는 다르지만 실질적으로 같은 상품인 경우
옵션 중복	동일 상품의 색상·용량 등 옵션만 다른 경우

9. 데이터 검증

AI Agent가 잘못된 상품 정보를 사용자에게 제공하지 않도록 수집 직후 검증을 수행한다.

9.1 필수 필드 검증

- 상품명 존재 여부
- 상품 URL 유효 여부
- 가격 숫자 형식 여부
- 통화 단위 확인
- 판매 상태 확인
- 수집 시각 확인

9.2 데이터 이상 탐지

- 가격이 0원 또는 음수인지 확인
- 정상가보다 판매가가 비정상적으로 높은지 확인
- 이전 수집 가격과 차이가 지나치게 큰지 확인
- 상품명과 카테고리가 일치하는지 확인
- 이미지 URL이 정상 응답하는지 확인

9.3 교차 검증

중요 상품은 다음 방식으로 추가 검증한다.

- 동일 사이트의 상품 목록과 상세 페이지 비교
- HTML 표시 가격과 JSON API 가격 비교
- 여러 판매처의 모델명 및 주요 사양 비교
- 이전 수집 데이터와 현재 데이터 비교
- AI Agent가 실제 상품 페이지를 다시 방문해 최종 정보 확인

10. 재수집 정책

상품 데이터는 가격, 재고, 배송 정보가 자주 변경되므로 항목별 수집 주기를 다르게 설정한다.

데이터	권장 수집 주기
상품명·브랜드	낮은 빈도
가격	높은 빈도
재고 상태	높은 빈도
배송 정보	중간 또는 높은 빈도
상품 설명·스펙	낮은 빈도
리뷰 수·평점	중간 빈도

변경이 자주 발생하거나 사용자 요청이 많은 상품은 우선순위를 높여 더 자주 수집한다.

11. 요청 제어 정책

대상 서버에 과도한 부하를 발생시키지 않도록 다음 정책을 적용한다.

- 사이트별 요청 속도 제한
- 동일 도메인 동시 요청 수 제한
- 요청 간격 설정
- 최대 재시도 횟수 제한
- 지수 백오프 적용
- 응답 타임아웃 설정
- robots.txt 확인 및 캐싱
- HTTP 429 응답 발생 시 요청 중단 또는 지연
- 수집 가능 시간대 설정

사이트별 제한 정책은 별도의 설정 파일로 관리한다.

```
sites:
  shop_a:
    request_interval_ms: 1500
    max_concurrency: 2
    timeout_seconds: 10
    retry_count: 3
    respect_robots_txt: true
```

12. 오류 처리

12.1 오류 유형

- HTTP 연결 실패
- 요청 타임아웃
- HTTP 403 또는 429 응답
- HTML 구조 변경
- JSON 응답 구조 변경
- 인증 토큰 만료
- JavaScript 렌더링 실패
- 필수 상품 데이터 누락
- 데이터베이스 저장 실패

12.2 처리 정책

오류	처리 방식
일시적 네트워크 오류	재시도
429 응답	대기 후 재시도 및 요청량 감소
403 응답	자동 반복 요청 중단 및 정책 확인
Selector 오류	파서 실패 기록 및 알림
JSON Schema 변경	필드 검증 실패 처리
저장 실패	메시지 큐 또는 재처리 저장소로 전달
반복 실패 URL	Dead Letter Queue에 저장

13. 모니터링

다음 지표를 수집하여 대시보드에서 확인한다.

- 사이트별 요청 수
- 수집 성공률
- 수집 실패율
- 평균 응답 시간
- HTTP 상태 코드 분포
- 파싱 성공률
- 필수 필드 누락률
- 중복 데이터 비율
- 상품 가격 변경 횟수
- 재시도 횟수
- 브라우저 인스턴스 수
- 수집 데이터 적재량

특정 사이트의 파싱 성공률이 기준치 이하로 떨어지면 사이트 구조 변경 가능성이 있으므로 알림을 발생시킨다.

14. 개발 단계

1단계: 대상 사이트 분석

- 수집 대상 사이트 선정
- robots.txt 및 이용 정책 확인
- 공식 API 제공 여부 확인
- HTML 구조 분석
- Network Fetch/XHR 요청 분석
- 인증 및 세션 필요 여부 확인

2단계: PoC 개발

- 사이트별 상품 100~1,000건 수집
- HTML 방식과 JSON 방식 비교
- 데이터 필드 추출 가능 여부 확인
- 수집 성공률 및 평균 처리 시간 측정
- 데이터 구조 변경 가능성 분석

3단계: 공통 수집 프레임워크 개발

- 사이트 Adapter 구조 구현
- 공통 Downloader 구현
- Parser 및 Normalizer 구현
- 데이터 검증 기능 구현
- 중복 처리 기능 구현

4단계: 운영 기능 개발

- 스케줄러 구현
- 재시도 및 실패 큐 구현
- 사이트별 요청 제한 적용
- 로그 및 모니터링 구축
- 파서 오류 알림 구현

5단계: AI Agent 연동

- 상품 검색 API 제공
- 상품 비교 데이터 제공
- 실시간 가격 및 재고 확인 기능 연동
- Agent 응답에 데이터 출처와 수집 시각 표시
- 결제 직전 상품 정보 재검증

15. 테스트 계획

15.1 단위 테스트

- HTML Selector 테스트
- JSON 필드 매핑 테스트
- 가격 문자열 변환 테스트
- URL 정규화 테스트
- 중복 판별 테스트

15.2 통합 테스트

- 대상 사이트 요청부터 DB 저장까지 전체 흐름 테스트
- 세션·쿠키가 필요한 사이트 테스트
- 페이지네이션 수집 테스트
- 무한 스크롤 테스트
- 옵션 선택 상품 테스트

15.3 장애 테스트

- 요청 타임아웃 상황
- HTTP 429 발생 상황
- HTML 구조 변경 상황
- JSON 필드 누락 상황
- 데이터베이스 연결 실패 상황
- 크롤러 프로세스 중단 후 재시작 상황

16. 성공 기준

PoC 단계의 성공 기준은 다음과 같이 설정한다.

- 대상 사이트별 핵심 상품 필드 수집 가능
- 상품 데이터 수집 성공률 95% 이상
- 필수 필드 누락률 5% 이하
- 동일 URL 중복 저장 방지
- 크롤링 실패 원인 추적 가능
- 사이트별 수집 방식 독립 적용 가능
- 상품 가격 및 재고 변경 이력 확인 가능
- AI Agent가 수집 데이터의 출처와 최신 수집 시각을 함께 제공 가능

17. 예상 위험 요소

위험 요소	대응 방안
사이트 HTML 구조 변경	사이트별 파서 분리 및 자동 테스트
API 응답 구조 변경	JSON Schema 검증 및 알림
접근 제한 발생	요청 속도 제한 및 정책 재검토
상품 정보 오수집	필수 필드 검증 및 교차 검증
중복 상품 증가	상품 식별 및 유사도 기반 중복 처리
동적 페이지 처리 비용 증가	JSON API 우선 적용
가격·재고 데이터 노후화	중요도 기반 재수집
특정 사이트 장애	사이트별 작업 격리
결제 전 정보 변경	결제 직전 실시간 재검증

18. 최종 산출물

- 크롤링 대상 사이트 분석서
- 사이트별 데이터 수집 명세서
- 사이트별 Adapter 구현 코드
- 공통 상품 데이터 Schema
- 크롤링 스케줄 정책
- 오류 및 재시도 정책
- 데이터 검증 규칙
- 모니터링 대시보드
- 테스트 결과서
- AI Agent 연동 API 명세서